Practical: 12
Date: 30/10/2026

Developing Multiple Linear Regression from Scratch

Procedure:

- Import the required Python libraries.
- Load the house size and price dataset.
- Display the total number of records and first five observations.
- Draw a scatter plot for house size and price.
- Perform feature normalization.
- Add a column of ones to the input data.
- Define the cost function and gradient descent.
- Apply gradient descent to estimate the model parameters.
- Plot the convergence of gradient descent.
- Compare different learning rates.
- Calculate and display actual and predicted prices.
- Calculate the R-squared score.
- Apply Linear Regression using Scikit-learn.
- Display the coefficients, intercept and model score.

Solution:

```
# import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Import Data
# Target & Feature Metric
df = pd.read_csv(r"C:\Users\Lenovo\Downloads\house_size_price_dataset.csv")
X = df.values[:, 0:2]
y = df.values[:, 2]
m = len(y)
print('Total no of training examples (m) = %s \n' %(m))

# Show only first 5 records
for i in range(5):
```

```
    print('x = ',X[i,],'y =',y[i])

# Scatter Plot
df.columns
X1 = df["size(in square feet)"]
Y1 = df[" price"]

plt.scatter(X1,Y1, color='red',marker= '+')
plt.grid()
plt.rcParams["figure.figsize"] = (10,6)
plt.xlabel('Size(in square feet)')
plt.ylabel('Price')
plt.title('Scatter plot of training data')
plt.show()


# Feature Normalization
def feature_normalize(X):
    mu = np.mean(X, axis = 0)
    sigma = np.std(X, axis= 0, ddof = 1)
    X_norm = (X - mu)/sigma
    return X_norm, mu, sigma
X, mu, sigma = feature_normalize(X)
print('mu= ', mu)
print('sigma= ', sigma)
print('X_norm= ', X[:5])

# Add column of ones to X feature
X = np.hstack((np.ones((m,1)), X))
X[:5]


# Compute Cost
def compute_cost(X, y, theta):
    predictions = X.dot(theta)
    errors = np.subtract(predictions, y)
    sqrErrors = np.square(errors)
    J = 1/(2 * m) * errors.T.dot(errors)
    return J


# Gradient Descent Function
def gradient_descent(X, y, theta, alpha, iterations):
    cost_history = np.zeros(iterations)
    for i in range(iterations):
```

```
        predictions = X.dot(theta)
        errors = np.subtract(predictions, y)
        sum_delta = (alpha / m) * X.transpose().dot(errors)
        theta = theta - sum_delta
        cost_history[i] = compute_cost(X, y, theta)
    return theta, cost_history

# Gradient Descent Parameters
theta = np.zeros(3)
iterations = 400
alpha = 0.15
theta, cost_history = gradient_descent(X, y, theta, alpha, iterations)
print('Final value of theta =', theta)
print('First 5 values from cost_history =', cost_history[:5])
print('Last 5 values from cost_history =', cost_history[-5:])

# Visualization of Convergence of Gradient Descent
plt.plot(range(1, iterations +1), cost_history, color ='blue')
plt.rcParams["figure.figsize"] = (10,6)
plt.grid()
plt.xlabel("Number of iterations")
plt.ylabel("cost (J)")
plt.title("Convergence of gradient descent")
plt.show()

# Effect of Learning Rate on Convergence
iterations = 400
theta = np.zeros(3)
alpha = 0.005
theta_1, cost_history_1 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.01
theta_2, cost_history_2 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.02
theta_3, cost_history_3 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.03
theta_4, cost_history_4 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.15
theta_5, cost_history_5 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 1.32
theta_6, cost_history_6 = gradient_descent(X, y, theta, alpha, iterations)


plt.plot(range(1, iterations +1), cost_history_1, color ='purple', label = 'alpha
= 0.005')
```

```
plt.plot(range(1, iterations +1), cost_history_2, color ='red', label = 'alpha =
0.01')
plt.plot(range(1, iterations +1), cost_history_3, color ='green', label = 'alpha
= 0.02')
plt.plot(range(1, iterations +1), cost_history_4, color ='yellow', label = 'alpha
= 0.03')
plt.plot(range(1, iterations +1), cost_history_5, color ='blue', label = 'alpha =
0.15')
plt.plot(range(1, iterations +1), cost_history_6, color ='brown', label = 'alpha
= 1.32')

plt.rcParams["figure.figsize"] = (10,6)
plt.grid()
plt.xlabel("Number of iterations")
plt.ylabel("cost (J)")
plt.title("Effect of Learning Rate On Convergence of Gradient Descent")
plt.legend()
plt.show()


# Actual vs Predicted
predict = np.round(X.dot(theta_5),1)
Act_Predict = np.stack((y, predict), axis=1)
print("Actual vs Predicted:")
print(Act_Predict[:5,])

# Calculating the r2 accuracy of our model
def r2score(predict, y):
    rss = np.sum((predict - y) ** 2)
    tss = np.sum((y-y.mean()) ** 2)
    r2 = 1 - (rss / tss)
    return r2
print("R2 Score:", np.round(r2score(predict, y),2))

# Sklearn
from sklearn import linear_model
model_ols = linear_model.LinearRegression()
model_ols.fit(X,y)
coef = model_ols.coef_
intercept = model_ols.intercept_
print('coef= ', coef)
print('intercept= ', intercept)
y_pred= model_ols.predict(X)
Act_Predict1 = np.stack((y, y_pred), axis=1)
```

```
print("Sklearn Actual vs Predicted:")
print(Act_Predict1[:5,])
print('Score: ', np.round(model_ols.score(X, y),2))
```

Output:

Total no of training examples (m) = 30

x = [650 1] y = 145000

x = [800 2] y = 182000

x = [900 2] y = 194000

x = [1000 2] y = 205000

x = [1100 2] y = 221000

mu= [1956.66666667 3.46666667]

sigma= [777.12260309 1.10588811]

X_norm= [[-1.68141637 -2.23048485]

[-1.48839663 -1.32623423]

[-1.35971681 -1.32623423]

[-1.23103699 -1.32623423]

[-1.10235716 -1.32623423]]

Final value of theta = [380333.33333333 122459.99897219 18603.34946026]

First 5 values from cost_history = [5.72411481e+10 4.03989808e+10 2.87151511e+10 2.05229075e+10

1.47321454e+10]

Last 5 values from cost_history = [13749413.04225202 13744174.12807214 13739015.14397147 13733934.87045744

13728932.10664313]

Actual vs Predicted:

[[145000. 132932.6]

[182000. 173391.9]

[194000. 189150.]

[205000. 204908.1]

[221000. 220666.3]]

R2 Score: 1.0

coef= [0.   124974.924259   16088.42417345]

intercept= 380333.3333333333

Sklearn Actual vs Predicted:

[[145000.   134313.46344518]

[182000.   172984.05781976]

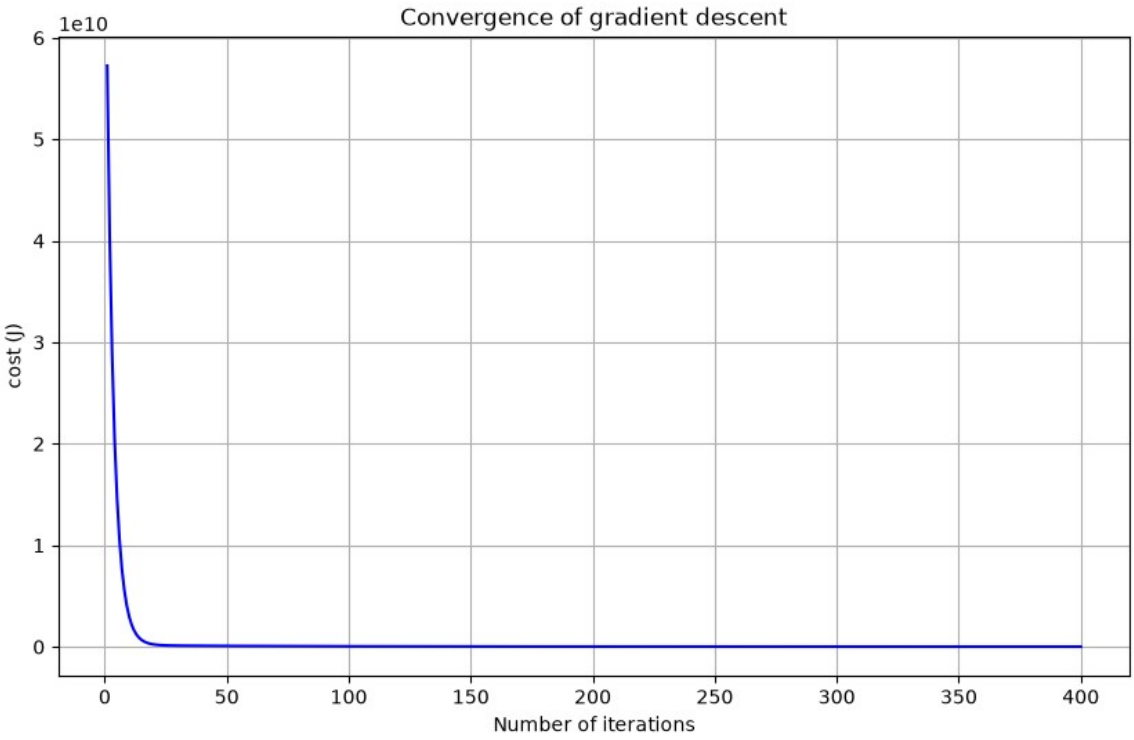
[194000.   189065.80911131]

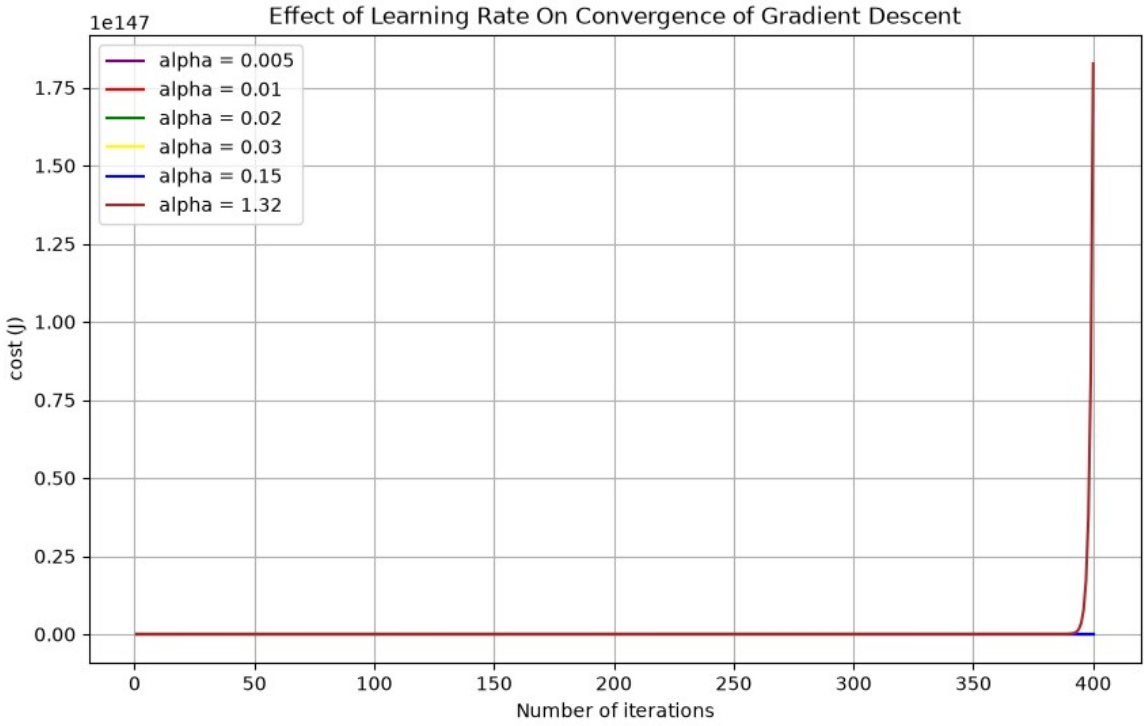
[205000.   205147.56040286]

[221000.   221229.31169441]]

Score: 1.0




Convergence of gradient descent
1e10
6
5
4
3
2
1
0
cost (J)
0
50
100
150
200
250
300
350
400
Number of iterations


Effect of Learning Rate On Convergence of Gradient Descent
1e147
alpha = 0.005
alpha = 0.01
alpha = 0.02
alpha = 0.03
alpha = 0.15
alpha = 1.32
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
cost (J)
0
50
100
150
200
250
300
350
400
Number of iterations